

From Bytes to Tensors: Efficient Media Processing with PyTorch in 2026

Nicolas Hug (META), Scott Schneider (META)

Plan

TorchCodec
decode & encode

TorchVision
image & video transforms

TorchAudio
audio transforms

Part 1: The landscape

Where these three libraries landed: who does what, and how they fit together

Part 2: Going fast

How to make your media pipeline fast with these three libraries, using existing and new features

All benchmarks: github.com/NicolasHug/ptc-talk-2026

Where the media libraries landed

TorchCodec

Decoding & encoding

images · video · audio · CPU & CUDA

TorchVision

Transforms

images & video

~~**Decoding & encoding**~~

→ **moved to TorchCodec**

Models, datasets

maintenance mode

TorchAudio

Transforms

audio

~~**Decoding & encoding**~~

→ **moved to TorchCodec**

Models, datasets, pipelines

maintenance mode

📖 Blog post: Evolution of the PyTorch media processing landscape

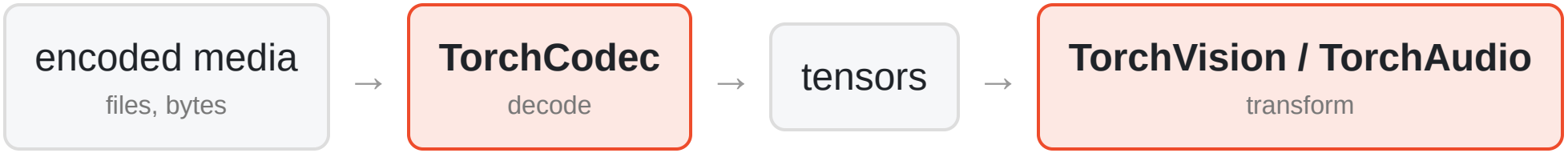
Why?

TorchCodec	TorchVision	TorchAudio
Decoding & encoding images · video · audio	**Transforms** images & video	**Transforms** audio

- **Clearer** for users
- **Clearer** for us: performance focus
- **Reduced maintenance** & dependencies
- **Transforms**: the most popular APIs
- **Models, datasets**: better served by the ecosystem

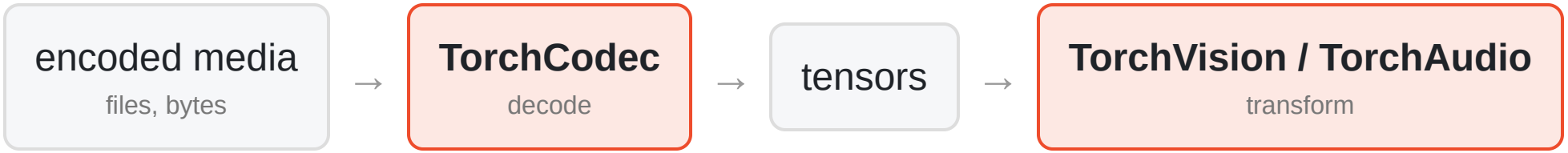
How they fit together

TorchCodec's **output** is TorchVision's and TorchAudio's **input**



How they fit together

TorchCodec's **output** is TorchVision's and TorchAudio's **input**



Video: TorchCodec → TorchVision

```python
import torch
from torchcodec.decoders import VideoDecoder
from torchvision.transforms import v2

clip = VideoDecoder("video.mp4")[10:20]
# uint8 tensor, (10, C, H, W)

transform = v2.Compose([
    v2.RandomResizedCrop(224),
    v2.ToDtype(torch.float32, scale=True),
])
batch = transform(clip)
```

Audio: TorchCodec → TorchAudio

```python
from torchcodec.decoders import AudioDecoder
from torchaudio.transforms import MelSpectrogram

decoder = AudioDecoder("audio.mp3")
samples = decoder.get_all_samples()

mel = MelSpectrogram(
    sample_rate=samples.sample_rate
)(samples.data)
```

Encoding goes the other way:

`VideoEncoder`, `AudioEncoder`, ...

TorchVision and TorchAudio are not deprecated

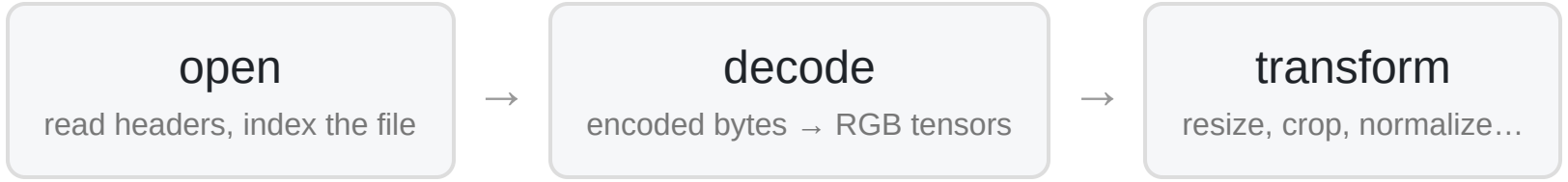
- They're **focused**: transforms are what the community uses most, so that's where the effort goes

All three libraries are now ABI stable

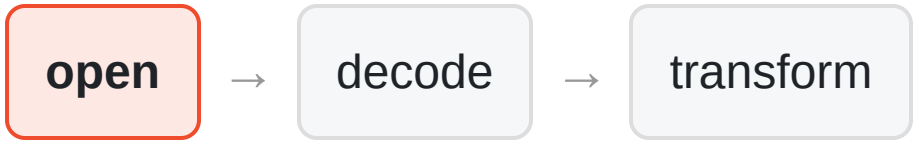
- A release is no longer tied to a single PyTorch version: it **keeps working with the PyTorch versions that come after it**
- No rebuild for every PyTorch release → release cadences no longer match PyTorch's, **and that's expected**

Still on `torchvision.io` / `torchaudio` decoding? → TorchCodec migration guide

Part 2: Going fast



Every stage can be the bottleneck. Let's go through them in order.



Opening the video

Open from anywhere

```
decoder = VideoDecoder("video.mp4")   # path, URL, bytes, tensor of bytes
```

But also **file-like objects.** Don't download the whole file!

```
import fsspec

with fsspec.open("https://.../video.mp4") as f:   # or s3://, gs://, ...
    decoder = VideoDecoder(f, seek_mode="approximate")
    frame = decoder[1000]
```

Only downloads the bytes around frame `1000`.

Exact vs approximate seek mode

```
decoder = VideoDecoder("long_video.mp4", seek_mode="approximate")
```

A tradeoff: **seek accuracy** vs **decoder creation speed**

	`"exact"` **(default)**	`"approximate"`
Creation	scans the whole file: slow on long videos	reads the header: fast
Seeking	frame-accurate	may be slightly off on some files

📊 Benchmark: remote 10 min 720p video (209 MB), create decoder + decode 16 frames

`"exact"` : **236 MB** downloaded

`"approximate"` : **28 MB** downloaded

Custom frame mappings: exact *and* fast

Pre-compute the index offline, reuse it every time you decode.

```
ffprobe -i video.mp4 -select_streams 0 -show_frames \
        -show_entries frame=pts,duration,key_frame -of json > mappings.json
```

```
with open("mappings.json") as f:
    decoder = VideoDecoder("video.mp4", custom_frame_mappings=f)
```

The best of both worlds:

- Seek accuracy like `"exact"`
- Creation speed like `"approximate"`



Decoding

GPU decoding: `device="cuda"`

```
decoder = VideoDecoder("video.mp4", device="cuda")
frames = decoder.get_frames_at([0, 10, 20])   # already on the GPU
print(decoder.cpu_fallback)                   # did we silently fall back to CPU?
```

Why it's (usually!) faster:

- **Fast raw decoding** on NVDEC, a dedicated hardware chip separate from the CUDA cores
- **Less PCIe traffic**: encoded packets are much smaller than decoded frames
- **YUV → RGB conversion**: expensive on CPU, extremely fast on GPU

📊 Benchmark: decode all 300 frames of a 10s video (laptop: RTX 4080)

	CPU	GPU	Speedup
1080p	1466 ms	370 ms	**4×**
4K	4394 ms	1430 ms	**3×**

On a warm cache!

📖 Tutorials: CUDA decoding · Image decoding

GPU decoding: the NVDEC decoder cache

Creating and destructing a `VideoDecoder(..., device="cuda")` is very expensive! TorchCodec **caches and reuses** the underlying hardware resources.

```
from torchcodec.decoders import set_nvdec_cache_capacity

set_nvdec_cache_capacity(64)   # default: 20 decoders per GPU
```

- A cached decoder is reused for a new video with the same codec and frame dimensions (LRU eviction)
- **tune the capacity!**

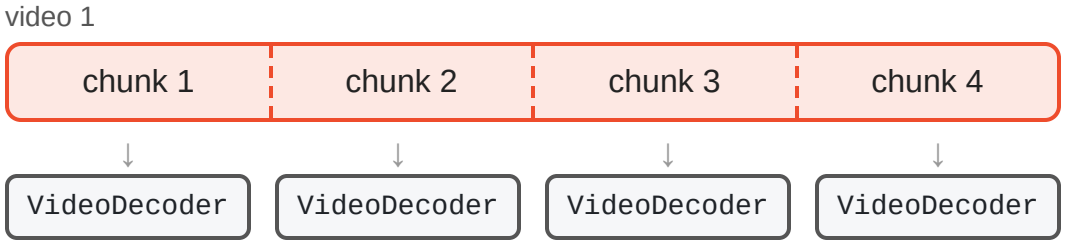
📊 Benchmark: create GPU decoder + decode 10 frames, 10s 1080p video (laptop: RTX 4080)

No cache	Warm cache	Speedup
272 ms	24 ms	**11×**

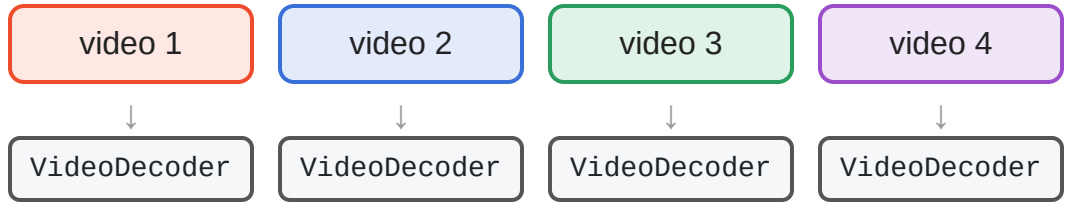
Matters even more when decoding many videos in parallel

Parallel decoding: 4 strategies

① One long video: one decoder per chunk



② Many videos: one decoder per video



③ FFmpeg threads: decode upcoming frames ahead, into a buffer

```
VideoDecoder(path, num_ffmpeg_threads=4)
```

④ Low-level APIs: parallelize stage by stage. Next slides!

All strategies compose! ① and ② run on threads or processes: decoding releases the GIL.

📊 Benchmark: decode all 18000 frames of a 10min 720p video (laptop CPU, 4 workers)

Sequential	① 4 threads	① 4 processes	③ 4 FFmpeg threads
32.7 s	8.7 s (**3.7×**)	9.3 s (**3.5×**)	15.3 s (**2.1×**)

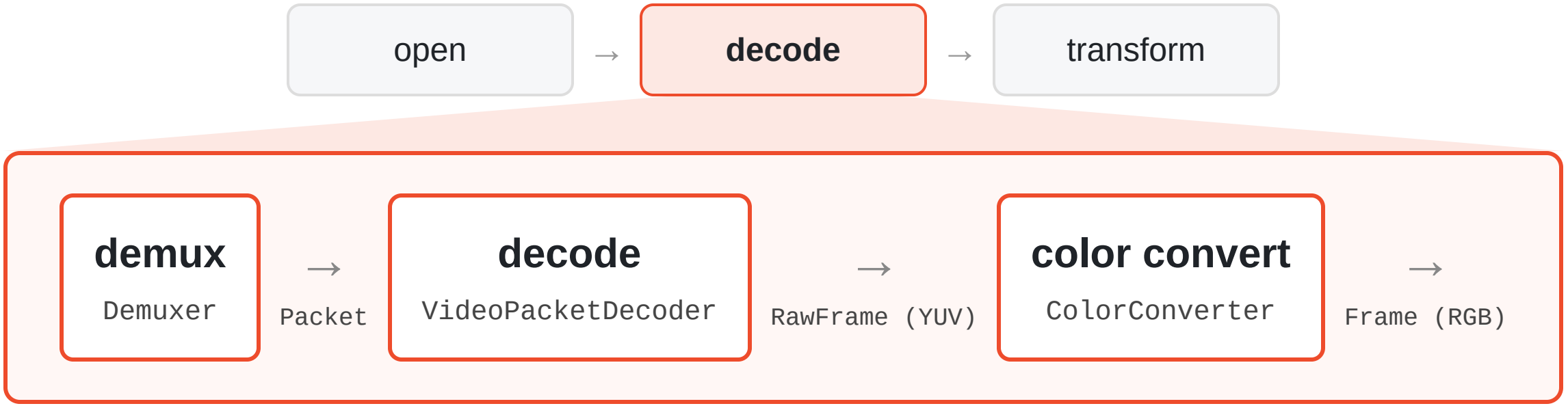
📖 Tutorial: Parallel video decoding

Low-level decoding APIs: new in TorchCodec 0.17



📖 Tutorial: Build your own decoding pipeline

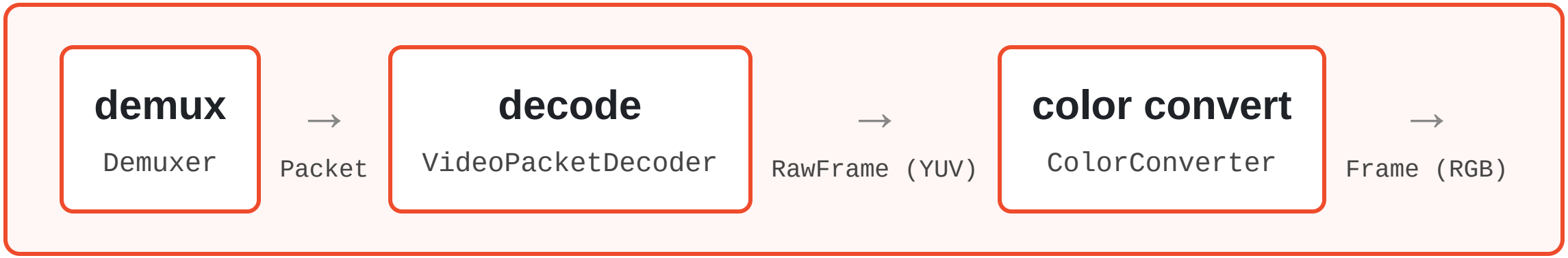
Low-level decoding APIs: new in TorchCodec 0.17



`VideoDecoder`: everything in one call. Low-level APIs: **more knobs to hill-climb on**.

📖 Tutorial: Build your own decoding pipeline

Low-level APIs: the basic loop




```python
from torchcodec.decoders import ColorConverter, Demuxer

demuxer = Demuxer("video.mp4", streams="video")
packet_decoder = demuxer.streams[0].make_decoder()
color_converter = ColorConverter()

for packet in demuxer:
    for raw_frame in packet_decoder.decode(packet):
        frame = color_converter.convert(raw_frame)

# Same as VideoDecoder("video.mp4").get_all_frames()
```

📖 Tutorial: Build your own decoding pipeline

What the low-level APIs enable

① **Multi-threaded pipelines**: run the stages concurrently

② **Multi-stream decoding**: audio + video in a single pass

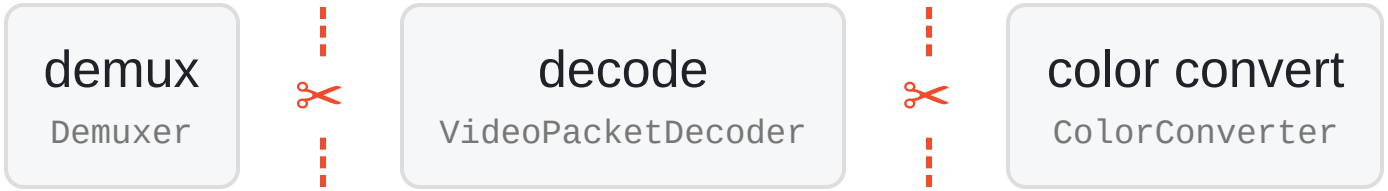
③ **Raw YUV frames**: custom color conversion, fused kernels, 10-bit HDR

Also: endless sources (live streams, pipes), cheap keyframe-only decoding

Let's go over ①, ② and ③

Low-level APIs: multi-threaded pipelines

Each stage **releases the GIL** → cut the pipeline into threads at any stage boundary:

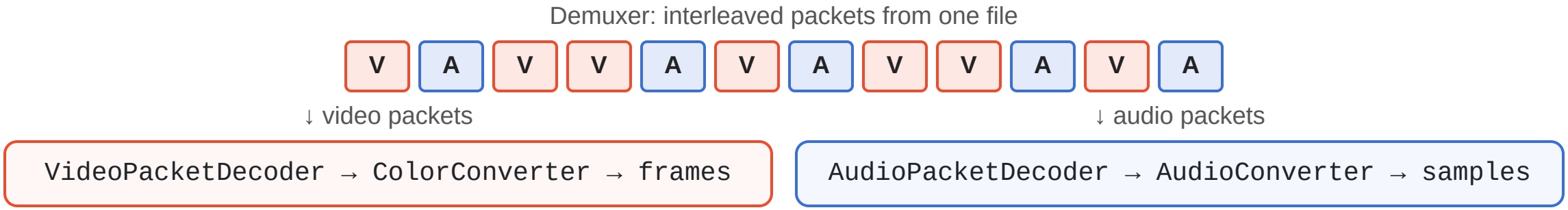



```
# prefetch(): run the upstream stages on a background thread, through a bounded queue
raw_frames = prefetch(decode(packet_decoder, demux(demuxer)))   # producer
frames = color_convert(color_converter, raw_frames)             # consumer
```

📊 Benchmark: decode all 300 frames of a 10s 1080p video (laptop CPU)

VideoDecoder	Low-level, sequential	Low-level, producer / consumer
1457 ms	1458 ms	940 ms (**1.5×**)

Low-level APIs: audio + video in one pass

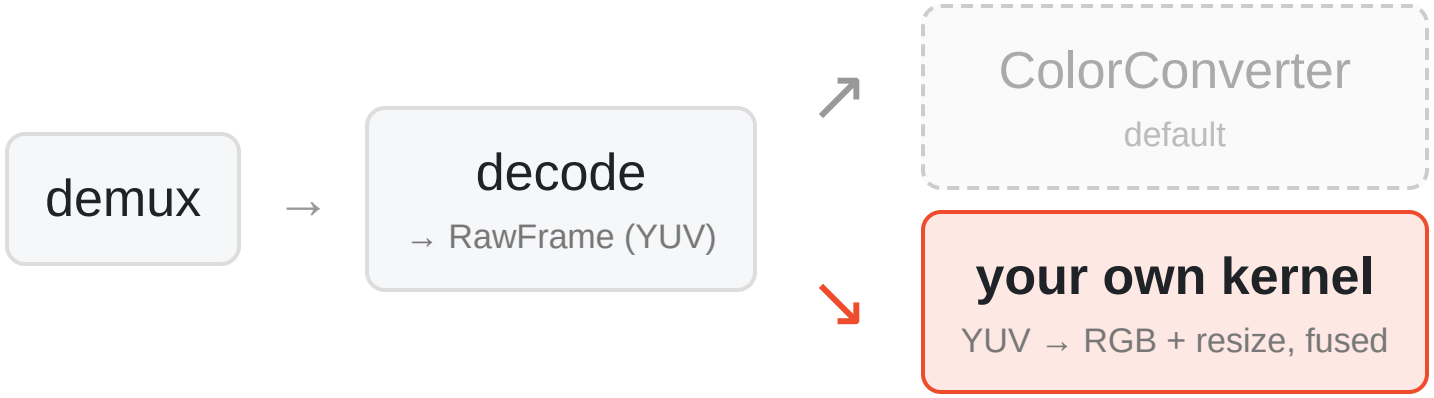



```
demuxer = Demuxer("video.mp4", streams=("video", "audio"))
video_stream, audio_stream = demuxer.streams
decoders = {s.index: s.make_decoder() for s in demuxer.streams}
color_converter, audio_converter = ColorConverter(), AudioConverter()

for packet in demuxer:
    for raw in decoders[packet.stream_index].decode(packet):
        if packet.stream_index == video_stream.index:
            frame = color_converter.convert(raw)
        else:
            samples = audio_converter.convert(raw)
```

- **Read the file once**, instead of once per stream
- Also works for several video streams or audio tracks: `streams="all"`

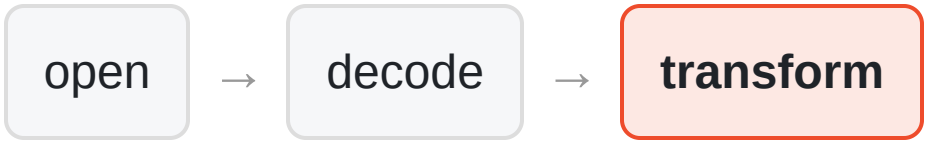
Low-level APIs: raw YUV frames




```
for raw_frame in packet_decoder.decode(packet):
    Y, U, V = raw_frame.planes        # the decoder's own planes: no conversion, no copy
    out = my_yuv_to_rgb_and_resize(Y, U, V, size=(224, 224))   # e.g. a Triton kernel
```

- **Fuse** color conversion with your transforms (e.g. resize) in one kernel, or train directly in YUV
- Or just train on YUV!
- 10-bit HDR → `uint16` planes, every bit intact

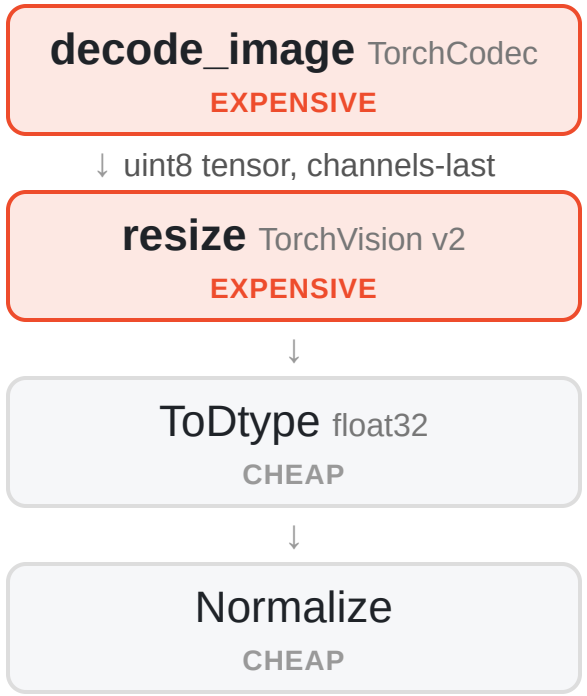
📖 Tutorial: Raw frames and raw audio samples



Transforming

(mostly images, but this applies to videos too!)

The bare minimum for an image pipeline



What fast resizing needs:

- **SIMD kernels** (AVX2, NEON), which need:
  - `uint8` input
  - `torchvision.transforms.v2`
  - **tensors**, not PIL images → TorchCodec image decoders
- **Channels-last** memory

TorchCodec + TorchVision give you all this by default!

📖 Docs: TorchVision transforms performance considerations

TorchCodec + TorchVision: fast by default

```python
from torchcodec.decoders import decode_image
from torchvision.transforms import v2

img = decode_image("image.jpg")             # uint8 CHW, channels-last memory

transforms = v2.Compose([                   # use v2!
    v2.Resize(size=(224, 224)),             # on uint8: SIMD fast path!
    v2.ToDtype(torch.float32, scale=True),  # convert to float *after* resizing
    v2.Normalize(mean=..., std=...),
])
out = transforms(img)
```

📊 Benchmark: decode + resize + normalize a 1080p JPEG (laptop CPU, 1 thread)

PIL pipeline	PIL-SIMD pipeline	TorchCodec + TorchVision (above)
10.0 ms	4.4 ms	**4.0 ms**

- PIL-SIMD: x86 only (no NEON SIMD), and still pays the `PIL → tensor` copy.
- `decode_image`: JPEG (CPU **and CUDA!**), PNG, WebP, GIF, AVIF, HEIC

Results are the same as PIL's!

- Use `antialias=True` (the default)
- vs PIL: mean absolute error **≈ 0.01** at most (out of 255), max difference **1-2**
- **Bitwise identical** to PIL-SIMD
- What matters for models: **consistent** interpolation mode and antialias between training and inference

NEW in `TorchVision 0.27`: **Lanczos interpolation!** Alongside bilinear and bicubic

Audio: `WavDecoder`

```
from torchcodec.decoders import WavDecoder

samples = WavDecoder("audio.wav").get_all_samples()
```

- Reads WAV data directly, **bypassing FFmpeg** entirely

📊 Benchmark: create decoder + decode a WAV file (laptop CPU)

	AudioDecoder	WavDecoder	Speedup
5s clip, 16kHz mono	2.7 ms	0.05 ms	**50×**
10min, 48kHz stereo	202 ms	81 ms	**2.5×**
1s from a 10min file	16.7 ms	0.06 ms	**280×**

And on the way out: encoding

```python
from torchcodec.encoders import Encoder

encoder = Encoder()
video = encoder.add_video(height=1080, width=1920, frame_rate=30, device="cuda")
audio = encoder.add_audio(sample_rate=44100, num_channels=2)

with encoder.open_file("output.mp4"):
    video.add_frames(frames)
    audio.add_samples(samples)
```

- Multiple video and audio streams in one container
- Mix CPU and CUDA video streams, feed data incrementally
- Simpler one-shot encoders too: `VideoEncoder`, `AudioEncoder`, `JpegEncoder`, `PngEncoder`

📖 Tutorials: Multi-stream encoding · Video encoding

Recap

Decoding / encoding: TorchCodec · **Transforms**: TorchVision, TorchAudio

Symptom	Fix
Videos in the cloud	File-like objects (e.g. `fsspec`)
Slow to open long videos	`seek_mode="approximate"` or custom frame mappings
CPU saturated by decoding, GPU starved	`device="cuda"`, tune `set_nvdec_cache_capacity()`, batched `decode_jpeg`
Decoding many frames / many videos	Threads or processes, `num_ffmpeg_threads`
Need more than `VideoDecoder` offers	Low-level APIs: multi-threaded pipelines, raw YUV access, multi-stream decoding
Slow image pipeline	TorchCodec `decode_image` → TorchVision v2 `resize` on `uint8` (SIMD)
Slow WAV decoding	`WavDecoder`

Feedback and feature requests welcome! TorchCodec · TorchVision · TorchAudio